A High-Performance Document Image Binarization Pipeline with Parallel Image Processing

Aghiad Khertabeel
aghiad.khertabeel@uni-jena.de
Friedrich Schiller University Jena
Jena, Germany

Mohamad Khaled Minawe
minawe.mohamad.khaled@uni-jena.de
Friedrich Schiller University Jena
Jena, Germany

ABSTRACT

This is a placeholder abstract for the paper.

KEYWORDS

document image binarization, image processing, OCR preprocessing, adaptive thresholding

1 INTRODUCTION

The digitization of documents has become increasingly prevalent in various domains, including libraries, archives and businesses. The goal is to revitalize the information contained in the physical documents and make it more accessible and searchable in digital form as well as preserving it for future generations. This process involves scanning physical documents in order to create a digital representation of them. However, the quality of the scanned images can vary significantly due to factors such as lighting conditions, document physical state and the scanning equipment used. As a result, the scanned images may contain noise, shadows, and other artifacts that can hinder the process of the recognition using OCR (Optical Character Recognition) systems [?].

To address such challenges, pre-processing techniques are often applied beforehand in order to enhance the quality of the scanned images and improve the accuracy of OCR systems [?]. One of the most common pre-processing techniques is image binarization, which converts a colorful or a greyscale image into a binary image consisting solely of black and white pixels [?]. According to Gupta et al., this process allows OCR systems to focus on the relevant information in the document and ignore the background, making the OCR systems more accurate [?].

While there are many algorithms for document or image binarization available, such as Niblack's approach and Sauvola's method, these algorithms can be computationally intensive as they often require processing each pixel or each cluster of pixels individually, which can be time-consuming, especially when working with large documents with high DPI [?, ?]. Therefore, there is a growing need for high-performance solutions that entail the usage of parallel processing techniques either on the CPU level or the GPU level. Furthermore, implementing parallelization on the CPU level can be achieved by splitting the work among many worker threads, whereas with GPU-level parallelization, the work can be distributed among many GPU threads, which are usually more efficient than CPU threads when it comes generally to image processing and rendering tasks [?]. This statement was shown by Ansari et al. as they did a performance comparison between parallelized CPU and GPU performance for large metrics and the GPU performance was consistently faster than the CPU by many folds. The reason for that is because GPUs tend to have significantly more cores than CPUs, allowing them to handle a larger amount of smaller computations in parallel.

2 BACKGROUND

3 SYSTEM ARCHITECTURE

The system implements a Scanned Document Enhancer designed for document quality improvement before printing or OCR processing. The architecture follows a modular C++ pipeline design with clear separation of concerns, enabling maintainable and testable code.

The processing utilizes sequential pipeline stages, allowing users to customize the enhancement workflow based on their specific needs.

OpenMP is also used for parallelization throughout compute-intensive operations to leverage multi-core CPU capabilities, significantly reducing processing time for large documents. The system uses the PPM P3 ASCII format for input and output images, which simplifies file handling and allows for easy debugging and visualization of intermediate results.

The design follows header/implementation separation, with clear interfaces defined in header files and implementation details encapsulated in source files. This promotes code reusability and maintainability, allowing easy extension of the system with additional enhancement techniques in the future.

3.1 Main Components

The main components can be seen in the directory structure in Listing ??:

The CLI layer, implemented in `main.cpp`, serves as the entry point of the application and the pipeline orchestration module. This layer deals with parsing arguments and configuration management.

The IO layer, located in the `io` directory, is responsible for reading and writing PPM images. It includes modules for handling the PPM P3 ASCII format, such as `ppm_reader` and `ppm_writer`, as well as a `token_reader`. The `ppm_reader` and `ppm_writer` modules handle the specifics of reading and writing PPM files, while the

`token_reader` module provides utility functions for parsing the ASCII format.

The Image Processing layer, located in the `image` directory, contains `GrayImage` as the main data structure for representing grayscale images. It contains the width, height, maxval, and flat vector data, along with colorspace utilities for converting between RGB and grayscale color spaces.

The Operations layer, located in the `ops` directory, contains various image enhancement techniques, including denoising (median filter), background estimation and removal, contrast stretching, thresholding methods (Otsu's, Sauvola's, Nick's, Su's, and a proposed method), morphology operations (open and close), and border cleanup. Each operation is implemented in its own module, allowing modularity and ease of maintenance.

Several operation modules utilize integral images for efficient computation of local statistics, while others rely on direct neighborhood processing. Additionally, OpenMP is used for parallelization to ensure high performance even when processing large documents.

The Utility Layer in util directory provides `args_parser` for command-line argument processing, logging for structured output and debugging, timing for performance measurement utilities, and clamp for mathematical helper functions. The Testing Framework in tests/ uses Catch2-based testing with 56 test cases and 1038 assertions, covering component testing for I/O and algorithms as well as integration testing, with validation for algorithm correctness and edge case handling.

Listing 1: Project directory structure

```
project/
|-- CMakeLists.txt
|-- data/
|   |-- in.ppm
|   `-- out.ppm
|-- src/
|   |-- main/
|   |   `-- main.cpp
|   |-- image/
|   |   |-- image_gray.h/.cpp
|   |   `-- colorspace.h/.cpp
|   |-- io/
|   |   |-- ppm_reader.h/.cpp
|   |   |-- ppm_writer.h/.cpp
|   |   `-- token_reader.h/.cpp
|   |-- ops/
|   |   |-- denoise_median.h/.cpp
|   |   |-- background_estimate.h/.cpp
|   |   |-- background_remove.h/.cpp
|   |   |-- contrast_stretch.h/.cpp
|   |   |-- threshold_otsu.h/.cpp
|   |   |-- threshold_sauvola.h/.cpp
|   |   |-- threshold_nick.h/.cpp
|   |   |-- threshold_su.h/.cpp
|   |   |-- threshold_proposed.h/.cpp
|   |   |-- morphology.h/.cpp
|   |   `-- border_cleanup.h/.cpp
|   `-- util/
|       |-- args_parser.h/.cpp
|       |-- logging.h/.cpp
|       |-- timing.h/.cpp
|       `-- clamp.h
`-- tests/
    |-- io/
    |   `-- test_ppm_io.cpp
    `-- ops/
        |-- test_otsu.cpp
        |-- test_sauvola.cpp
        `-- test_median.cpp
```

3.2 Data Flow

The high-level pipeline follows the sequence:

(1) The user invokes the application via the command line, providing input and output file paths along with optional parameters for the enhancement pipeline.
(2) The CLI layer parses the arguments and configures the processing pipeline accordingly.
(3) The IO layer reads the input PPM image and converts it to a `GrayImage` format for processing.
(4) The Image Processing layer applies the selected enhancement operations sequentially, with each operation taking the output of the previous one as input. Operations may utilize integral images for efficient local computations and OpenMP for parallel processing.
(5) After processing, the final enhanced image is written back to a PPM file by the IO layer.

The detailed processing steps begin with the Input Stage performing PPM P3 file parsing and validation. Color Conversion transforms RGB to grayscale using standard luminance weights. The Preprocessing Chain includes optional median denoising with a 3×3 kernel, background estimation via separable box blur, background removal with target brightness adjustment, and percentile-based contrast stretching.

The Binarization Stage applies one of five configurable methods to the preprocessed image. The Postprocessing Chain performs optional morphological operations using a 3×3 structuring element and border cleanup with either fixed width or adaptive dark pixel removal. The Output Stage writes the final binary image as PPM P3 format.

Memory Management utilizes single-pass processing to minimize memory footprint, in-place operations where algorithmically feasible, integral images precomputed for adaptive thresholding methods, and OpenMP parallelization with thread-local variables and reduction operations. This architecture ensures modularity, testability, and performance while maintaining code clarity and extensibility.

4 IMPLEMENTED BINARIZATION METHODS

as is have been said before the system implements 5 different binarization methods, Otsu's, Sauvola's, Nick's, Su's, and a proposed method. The first three methods are implemented as described in their respective papers, while on the su method we have implemented the core idea of the method but with some modifications.

REFERENCES

[1] Mufakir Qamar Ansari and Mudabir Qamar Ansari. Accelerating matrix multiplication: A performance comparison between multi-core cpu and gpu, 2025.

[2] Ankush Mittal Debashish Ghosh Brij Mohan Singh, Rahul Sharma. Parallel implementation of niblackís binarization approach on cuda. *International Journal of Computer Applications*, 32(2):22–27, October 2011.

[3] Xin Chen, Yuefang Gao, and Zhonghong Huang. Cuda-accelerated fast sauvola's method on kepler architecture. *Multimedia Tools Appl.*, 74(24):11809–11820, December 2015.

[4] Maya R. Gupta, Nathaniel P. Jacobson, and Eric K. Garcia. Ocr binarization and image pre-processing for searching historical documents. *Pattern Recognition*, 40(2):389–397, 2007.

[5] Mohamed Ali Souibgui and Yousri Kessentini. De-gan: A conditional generative adversarial network for document enhancement. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1180–1191, March 2022.

[6] Chris Tensmeyer and Tony Martinez. Historical document image binarization: A review. *SN Comput. Sci.*, 1(3), May 2020.

A RESEARCH METHODS

A.1 Part One

Placeholder for Part One.

A.2 Part Two

Placeholder for Part Two.

B ONLINE RESOURCES

Placeholder for Online Resources.